CITS2200 Data Structures and Algorithms

Class Test 1 - 25/3/2026

Instructions:

- **Please fill in your student number (and if possible name) correctly in the answer sheet.**
- **Total marks for this paper is 10.**
- **Answer all questions by filling the correct choice in the answer sheet.**
- **Please do not return the question paper.**

Q1. What is the complexity of merging two sorted arrays of total size n? [cite: 108]

(a) $O(n)$

(b) $O(n \log n)$

(c) $O(\log n)$

(d) $O(n^2)$

Answer: (a)

Explanation: To merge two sorted arrays, you only need to traverse each array once, comparing the front elements and moving them into a new array. This linear scan takes $O(n)$ time.

Q2. Consider the following code:

```
for i in range(n):
    for j in range(n):
        if j < i:
            print(i,j)
```

What is the time complexity of this code? [cite: 109]

(a) $O(n)$

(b) $O(n \log n)$

(c) $O(n^2)$

(d) $O(n^3)$

Answer: (c)
Explanation: The code consists of two nested loops, each iterating up to n times. The total number of iterations is roughly $n^2/2$, which simplifies to $O(n^2)$.

Q3. Binary search is applied to the array

$$[3, 7, 12, 18, 21, 30, 42]$$

Searching for the target element 21. [cite: 110] What is the **first element** compared with the target? [cite: 110]

(a) 3

(b) 12

(c) 18

(d) 21

Answer: (c)
Explanation: Binary search begins by comparing the target with the middle element. In an array of size 7, the middle index is 3 (using 0-indexing), which corresponds to the value 18.

Q4. Suppose you have to sort an array in ascending order using **insertion sort**. [cite: 111] The input array is in **descending order**. What is the worst-case time complexity? [cite: 111]

(a) $O(n)$

(b) $O(n \log n)$

(c) $O(n^2)$

(d) $O(2^n)$

Answer: (c)
Explanation: When an array is in reverse (descending) order, each new element must be compared and shifted past all previously sorted elements, resulting in $O(n^2)$ comparisons and swaps.

Q5. Which of the following is asymptotically smallest? [cite: 113]

(a) n

(b) $n \log n$

(c) n^2

(d) 2^n

Answer: (a)
Explanation: In terms of growth rates, linear growth (n) is smaller than linearithmic ($n \log n$), quadratic (n^2), and exponential (2^n) growth.

Q6. Suppose merge sort is applied to an array of size n. [cite: 114] How many recursive subproblems of size $n/2$ are created in the first step? [cite: 114]

(a) 1

(b) 2

(c) $\log n$

(d) n

Answer: (b)
Explanation: Merge sort follows a divide-and-conquer approach where the array is split into exactly two halves (subproblems) in the first recursive step.

Q7. A sorted array has size n. We perform k binary searches on it. [cite: 116] What is the total complexity? [cite: 116]

(a) $O(k)$

(b) $O(k \log n)$

(c) $O(nk)$

(d) $O(n \log k)$

Answer: (b)
Explanation: A single binary search on an array of size n takes $O(\log n)$ time. Performing this k times results in a total complexity of $O(k \log n)$.

Q8. Suppose an algorithm performs the following steps:

- sort the array using merge sort
- scan the sorted array once

What is the overall complexity? [cite: 117]

(a) $O(n)$

(b) $O(n \log n)$

(c) $O(n^2)$

(d) $O(\log n)$

Answer: (b)
Explanation: Sorting takes $O(n \log n)$ and the scan takes $O(n)$. Since $O(n \log n)$ dominates $O(n)$, the overall complexity is $O(n \log n)$.

Q9. Which of the following is **not** required for binary search? [cite: 118]

(a) Random access to the array

(b) The array must be sorted

(c) Ability to compare elements

(d) The array must contain distinct elements

Answer: (d)
Explanation: Binary search works perfectly fine on arrays with duplicate elements; it will simply return one of the occurrences of the target value.

Q10. What is the minimum number of comparisons required to find the maximum element in an array of n numbers? [cite: 119]

(a) $n - 1$

(b) $\log n$

(c) n

(d) $n \log n$

Answer: (a)
Explanation: To find the maximum, you must compare the current maximum with every other element in the array. For n elements, this requires $n - 1$ comparisons.